

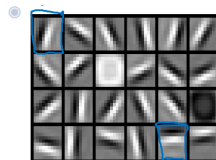
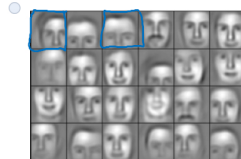
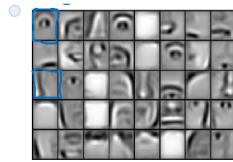
1. What is the "cache" used for in our implementation of forward propagation and backward propagation? 1 point

- ☐ It is used to cache the intermediate values of the cost function during training.
- ☐ It is used to keep track of the hyperparameters that we are searching over, to speed up computation.
- ☐ We use it to pass variables computed during backward propagation to the corresponding forward propagation step. It contains useful values for forward propagation to compute activations.
- ☒ We use it to pass Z computed during forward propagation to the corresponding backward propagation step. It contains useful values for backward propagation to compute derivatives.

2. During the backpropagation process, we use gradient descent to change the hyperparameters. True/False? 1 point

- ☐ False
- ☒ True

3. Which of the following is more likely related to the early layers of a deep neural network? 1 point



4. Vectorization allows you to compute forward propagation in an L -layer neural network without an explicit for-loop (or any other explicit iterative loop) over the layers $l=1, 2, \dots, L$. True/False? 1 point

- ☒ True
- ☐ False

5. Assume we store the values for $n_l^{(0)}$ in an array called `layer_dims`, as follows: `layer_dims = [n_L, 4, 3, 2, 1]`. So layer 1 has four hidden units, layer 2 has 3 hidden units and so on. Which of the following for-loops will allow you to initialize the parameters for the model? 1 point

- ☐

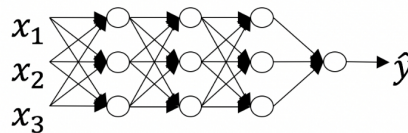
```
for i in range(1, len(layer_dims)/2):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```
- ☒

```
for i in range(1, len(layer_dims)):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```
- ☐

```
for i in range(1, len(layer_dims)/2):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i-1], 1) * 0.01
```
- ☐

```
for i in range(1, len(layer_dims)):
    parameter['W' + str(i)] = np.random.randn(layer_dims[i-1], layer_dims[i]) * 0.01
    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```

6. Consider the following neural network. 1 point



How many layers does this network have?

- ☐ The number of layers L is 4. The number of hidden layers is 4.
- ☐ The number of layers L is 3. The number of hidden layers is 3.
- ☐ The number of layers L is 5. The number of hidden layers is 4.
- ☒ The number of layers L is 4. The number of hidden layers is 3.

7. During forward propagation, in the forward function for a layer l you need to know what is the activation function in a layer (sigmoid, tanh, ReLU, etc.). During backpropagation, the corresponding backward function also needs to know what is the activation function for layer l , since the gradient depends on it. True/False? 1 point

- ☐ True
- ☒ False

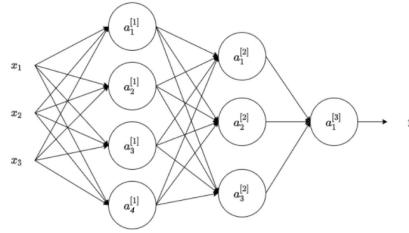
8. There are certain functions with the following properties: 1 point

(i) To compute the function using a shallow network circuit, you will need a large network (where we measure size by the number of logic gates in the network), but (ii) To compute it using a deep network circuit, you need only an exponentially smaller network. True/False?

- ☒ True
☐ False

9. Consider the following 2 hidden layers neural network:

1 point



Which of the following statements is true? (Check all that apply).

- ☐ $W^{[1]}$ will have shape (3, 4)
☐ $W^{[2]}$ will have shape (3, 1)
☐ $W^{[2]}$ will have shape (4, 3)
☒ $W^{[1]}$ will have shape (4, 3)
☐ $W^{[2]}$ will have shape (1, 3)
☐ $b^{[1]}$ will have shape (1, 4)
☒ $b^{[1]}$ will have shape (4, 1)
☐ $b^{[1]}$ will have shape (3, 1)
☒ $W^{[2]}$ will have shape (3, 4)

10. Whereas the previous question used a specific network, in the general case what is the dimension of $W^{[l]}$, the weight matrix associated with layer l ?

1 point

- ☐ $W^{[l]}$ has shape $(n^{[l+1]}, n^{[l]})$
☒ $W^{[l]}$ has shape $(n^{[l]}, n^{[l-1]})$
☐ $W^{[l]}$ has shape $(n^{[l-1]}, n^{[l]})$
☐ $W^{[l]}$ has shape $(n^{[l]}, n^{[l+1]})$